

Introduction to Artificial Intelligence

Search I

Matteo Acclavio

19/02/2026

Plan

So far:

- We have seen how to describe an agent and its environment
- We have seen how to classify environments and agents
- We have seen some simply reflex agents

Today:

- We will see how to design agents that can plan their actions
- We will see how to design agents that can search for a solution

Solving-problems agents

Definition

A *solving-problems agent* is an agent that can *plan ahead* actions.


search

Solving-problems agents

Definition

A *solving-problems agent* is an agent that can *plan ahead* actions.



We focus on 'simple environments':

- Single-agent;
- Fully observable and Known;
- Deterministic;
- Sequential;
- Static;
- Discrete.

Solving-problems agents

Definition

A ***solving-problems agent*** is an agent that can *plan ahead* actions.


search

We focus on 'simple environments':

- Single-agent;
- Fully observable and Known;
- Deterministic;
- Sequential;
- Static;
- Discrete.

We see two types of search algorithms:

- ① Uninformed search: agent cannot estimate the 'distance' to the goal
- ② Informed search

Problem-solving Phases

To solve the problem, the problem-solving agent goes through the following phases:

- ➊ **Goal definition:** what is the goal of the agent?
- ➋ **Problem formulation:** what are the states and actions of the problem space?
- ➌ **Search:** what is the sequence of actions that leads to the goal?
- ➍ **Execution:** execute the plan!

Definition:

A ***solution*** to a problem is a sequence of actions that leads from the initial state to a goal state.

Remarks

- In a fully-observable, deterministic, known environment, the agent can plan its actions by searching for a solution in the problem space



[Open loop system]

Remarks

- In a fully-observable, deterministic, known environment, the agent can plan its actions by searching for a solution in the problem space



[Open loop system]

- Otherwise, we may need to consider percepts to plan the actions [Closed loop system]. In this case, let the solution have non-determinism, and update the plan based on the percepts received during execution.

Problem Formulation (and Goal definition)

We assume states are 'atomic'.

- *set of states* S
- *initial state* $s_0 \in S$
- *goal states* $G \subseteq S$
- *actions* $\mathcal{A}$
- *available actions* $A : S \rightarrow \mathcal{A}(-) \subset \mathcal{A}$
- *transition model* $T : S \times \mathcal{A} \rightarrow S$
- *action cost function* $C : S \times \mathcal{A} \times S \rightarrow \mathbb{R}^+$

A *path* is a sequence of (states and) actions.

A *solution* is a path starting from the initial state and ending in a goal.

Example I: Vacuum Cleaner

- *set of states*
- *initial state*
- *goal states*
- *actions*
- *available actions*
- *transition model*
- *action cost function*

Example II: Water Jug Riddle

- *set of states*
- *initial state*
- *goal states*
- *actions*
- *available actions*
- *transition model*
- *action cost function*



You have a 3-liter and a 5-liter jug, and an unlimited water. How can you measure exactly 4 liters of water?

Example III: $3n + 1$ Problem

- *set of states*
- *initial state*
- *goal states*
- *actions*
- *available actions*
- *transition model*
- *action cost function*

Given a number n ,
if n is even, divide it by 2,
otherwise multiply it by 3 and add 1.
Does this process eventually reach the number 1?

Example IV: Route Finding

- *set of states*
- *initial state*
- *goal states*
- *actions*
- *available actions*
- *transition model*
- *action cost function*

Search Algorithms

Search Algorithms

- We represent the problem as a graph
- We can then search a solution by searching a path.

Search Algorithms

- We represent the problem as a graph
- We can then search a solution by searching a path.

Tree search:

- node = state
- edge = action
- root = initial state
- children = states reachable from the parent state by an action
- parent = state from which the current state is reached
- frontier = set of nodes available for expansion*
- explored set = set of states already explored

*how we represent the frontier depends on the search algorithm we use.

Evaluating Algorithms

We evaluate algorithms based on:

- **Completeness:** does the algorithm always find a solution if one exists?
- **Time complexity:** how long does it take to find a solution?
- **Space complexity:** how much memory does it use?
- **Optimality:** does the algorithm always find the optimal solution?

Uninformed Search Algorithms

Uninformed

- Breadth-First Search
- Dijkstra's algorithm (aka Uniform Cost Search, or 'Cheap First')
- Depth-First Search
- Depth-Limited Search
- Iterative Deepening Search

Breadth-First Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

Breadth-First Search

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness:

Breadth-First Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity:

Breadth-First Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: $O(n^d)$
- Space complexity:

Breadth-First Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: $O(n^d)$
- Space complexity: $O(n^d)$
- Optimal?

Breadth-First Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: $O(n^d)$
- Space complexity: $O(n^d)$
- Optimal? Only if all actions have the same cost

where n is the branching factor (number of children of each node) and d is the depth of the first solution.

Uniform Cost Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

Uniform Cost Search

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness:

Uniform Cost Search

```
1: frontier ← queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity:

Uniform Cost Search

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: better than breadth-first search
- Space complexity:

Uniform Cost Search

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: better than breadth-first search
- Space complexity: better than breadth-first search
- Optimal?

Uniform Cost Search

```
1: frontier  $\leftarrow$  queue containing the initial state
2: while frontier is not empty do
3:   order frontier by path cost, and node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: yes
- Time complexity: better than breadth-first search
- Space complexity: better than breadth-first search
- Optimal? Depends on the cost function

Depth-First Search

```
1: frontier  $\leftarrow$  stack containing the initial state
2: while frontier is not empty do
3:   node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

Depth-First Search

```
1: frontier  $\leftarrow$  stack containing the initial state
2: while frontier is not empty do
3:   node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness:

Depth-First Search

```
1: frontier  $\leftarrow$  stack containing the initial state
2: while frontier is not empty do
3:   node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: NO (can get stuck in infinite branches)
- Time complexity:

Depth-First Search

```
1: frontier ← stack containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: NO (can get stuck in infinite branches)
- Time complexity: $O(n^m)$
- Space complexity:

Depth-First Search

```
1: frontier  $\leftarrow$  stack containing the initial state
2: while frontier is not empty do
3:   node  $\leftarrow$  frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: NO (can get stuck in infinite branches)
- Time complexity: $O(n^m)$
- Space complexity: $O(nm)$
- Optimal?

Depth-First Search

```
1: frontier ← stack containing the initial state
2: while frontier is not empty do
3:   node ← frontier.pop()
4:   if node is a goal state then
5:     return solution
6:   else
7:     add node's children to frontier
8:   end if
9: end while
```

- Completeness: NO (can get stuck in infinite branches)
- Time complexity: $O(n^m)$
- Space complexity: $O(nm)$
- Optimal? NO

where m is the maximum depth of the search tree

Depth-First Search variants

- *limited depth-first search*: depth-first search with a limit on the depth of the search tree
- *iterative deepening search*: depth-first search with increasing depth limits

Comparing algorithms

Algorithm	Completeness	Time complexity	Space complexity	Optimal?
Breadth-First	Yes	$O(n^d)$	$O(n^d)$	Depends on the cost function
Uniform Cost	Yes	Better than BFS	Better than BFS	Depends on the cost function
Depth-First	No	$O(n^m)$	$O(nm)$	No
Depth-Limited	No	$O(n^\ell)$	$O(n^\ell)$	No
Iterative Deepening	Yes	$O(n^d)$	$O(nd)$	Depends on the cost function

Direction of the search

So far, we have seen algorithms that search from the initial state to the goal state (forward search).

More in general, we can have the following approaches:

- ***forward search***: search from the initial state to the goal state
- ***backward search***: search from the goal state to the initial state
- ***bidirectional search***: search from both the initial state and a goal state, and meet in the middle

Summary

Book references:

- Chapter 3 [uninformed = until 3.4 included]

Next TA session:

- Exercises on Search

Next lecture:

- Search in Complex Environments